Universidad del Valle de Guatemala

Campus Sur · Facultad de Ingeniería

Licenciatura en Tecnología de Sistemas Informáticos

Proyecto de Desarrollo de Software Seguro

Secure SDLC aplicado al sistema FORJA

Curso: Seguridad de Sistemas Informáticos

Catedrático: Ing. Rolando Iván Chalí Monroy

Integrantes

José Julián Amado García · 20968

Jorge Luis de la Rosa Zamora · 20499

Imri Gabriel Contreras Bautista · 20926

Guatemala, noviembre de 2026

Control de versiones del documento

Versión 1.0. Estructura del documento y escaneo SAST inicial con Snyk.

Versión 1.1. Remediaciones SAST (dependencias y cookies) y modelo de amenazas v1.1.

Versión 1.2. Análisis DAST con OWASP ZAP, cabeceras de seguridad, modelo de amenazas v1.2 y versión final.

Las tres versiones se elaboraron entre el 4 y el 5 de noviembre de 2026, por lo que se identifican por fase del proyecto y no por fecha.

Índice

1. Introducción.....3
1.1 Objetivo del proyecto.....3
1.2 Sistema evaluado.....3
1.3 Alcance.....3
1.4 Entregables y metodología.....4
1.5 Equipo y responsabilidades.....4
1.6 Resumen de lo realizado.....4
2. Repositorio de documentación.....5
3. Modelo de amenazas.....6
3.1 Descripción y diagrama de flujo de datos.....6
3.2 Análisis de la superficie de ataque.....7
3.3 Amenazas identificadas, categorización y priorización.....8
3.4 Versionado del modelo de amenazas.....11
3.5 Configuración segura para el despliegue (documentada, no aplicada).....12
4. Análisis estático (SAST).....13
4.1 Herramienta y configuración.....13
4.2 Resultados del escaneo inicial.....13
4.3 Detalle de vulnerabilidades.....14
4.4 Remediaciones.....19
4.5 Escaneo posterior a las correcciones.....21
5. Análisis dinámico (DAST).....23
5.1 Herramienta y configuración.....23
5.2 Resultados del escaneo.....23
5.3 Detalle de vulnerabilidades.....24
5.4 Correcciones y reescaneo.....26
6. Relación con el modelo de amenazas.....29
7. Conclusiones y lecciones aprendidas.....30
Anexo A. Hallazgos bajos del código fuente (Snyk Code).....31
Anexo B. Glosario.....32
Anexo C. Evidencias.....33

1. Introducción

1.1 Objetivo del proyecto

Aplicar buenas prácticas de desarrollo seguro (Secure SDLC) sobre un sistema real, siguiendo las fases del framework y documentando las medidas tomadas: modelado de amenazas, análisis estático (SAST), análisis dinámico (DAST) y remediación de las vulnerabilidades encontradas.

1.2 Sistema evaluado

El sistema evaluado es FORJA, una aplicación web de generación y monitoreo de rutinas de entrenamiento personalizadas basada en reglas de periodización y análisis de rendimiento. Corresponde al trabajo de graduación de uno de los integrantes del equipo.

Aspecto	Detalle
Backend	Laravel 13 (PHP 8.3), ejecutado en local con Laravel Herd
Frontend	Inertia 3 + React 19 con TypeScript, compilado con Vite 8
Base de datos	MySQL
Autenticación	Laravel Fortify: sesión con driver database, autenticación de dos factores y passkeys
Roles	Usuario y administrador
Repositorio	github.com/josejulianamadogarcia10-lab/forja (repositorio privado)
Ambiente de pruebas	Local (localhost), sin datos reales

1.3 Alcance

Se evalúa el código fuente, las dependencias y la aplicación en ejecución dentro de un ambiente local de pruebas. No se realizaron pruebas contra sistemas de terceros ni contra ambientes productivos.

1.4 Entregables y metodología

Entregable	Sección	Herramienta
Repositorio de documentación	2	GitHub
Modelo de amenazas y análisis de superficie de ataque	3	Microsoft Threat Modeling Tool
Análisis SAST y remediaciones	4	Snyk (Code y Open Source)
Análisis DAST	5	OWASP ZAP

1.5 Equipo y responsabilidades

El equipo no estableció roles fijos: los tres integrantes participaron en todas las fases del proyecto (modelo de amenazas, SAST, DAST y documentación) y pueden explicar todo el proceso.

Integrante	Participación
José Julián Amado García	Todas las fases
Jorge Luis de la Rosa Zamora	Todas las fases
Imri Gabriel Contreras Bautista	Todas las fases

1.6 Resumen de lo realizado

- Qué se revisó: el código fuente, las dependencias de npm y Composer, y la aplicación en ejecución con dos cuentas de prueba (usuario y administrador).
- Qué se encontró: 31 hallazgos de Snyk (10 en dependencias y 21 en código) y 17 alertas de OWASP ZAP, ninguna de riesgo alto.
- Qué se cambió: se actualizaron concurrently, vite, league/commonmark y symfony/http-foundation; se agregó el atributo Secure a dos cookies; se creó un middleware de cabeceras de seguridad (CSP, X-Frame-Options y nosniff) y se retiró X-Powered-By.
- Con qué evidencia se comprobó: el reescaneo de Snyk quedó con 0 hallazgos abiertos; el de ZAP no reporta clickjacking y CSP solo falta en respuestas de error; las 41 pruebas automáticas pasan y el frontend compila sin errores.

2. Repositorio de documentación

Toda la documentación del proyecto se encuentra en el siguiente repositorio: https://github.com/josejulianamadogarcia10-lab/forja-secure-sdlc-docs.

Estructura del repositorio:

Carpeta	Contenido
/threat-model	Archivos .tm7 del modelo de amenazas por versión (v1.0, v1.1, …) y reportes exportados
/sast	Reportes de Snyk antes y después de las correcciones
/dast	Reportes de OWASP ZAP antes y después de las correcciones
/evidencias	Capturas de pantalla con fecha
/documento-final	Este documento en Word y PDF
README.md	Índice del repositorio y descripción del proyecto

3. Modelo de amenazas

3.1 Descripción y diagrama de flujo de datos

El modelo se elaboró con Microsoft Threat Modeling Tool a partir de un diagrama de flujo de datos (DFD) que representa a los actores, procesos, almacén de datos, flujos y fronteras de confianza de FORJA, un sistema web de rutinas de entrenamiento construido con Laravel, Inertia/React y MySQL.

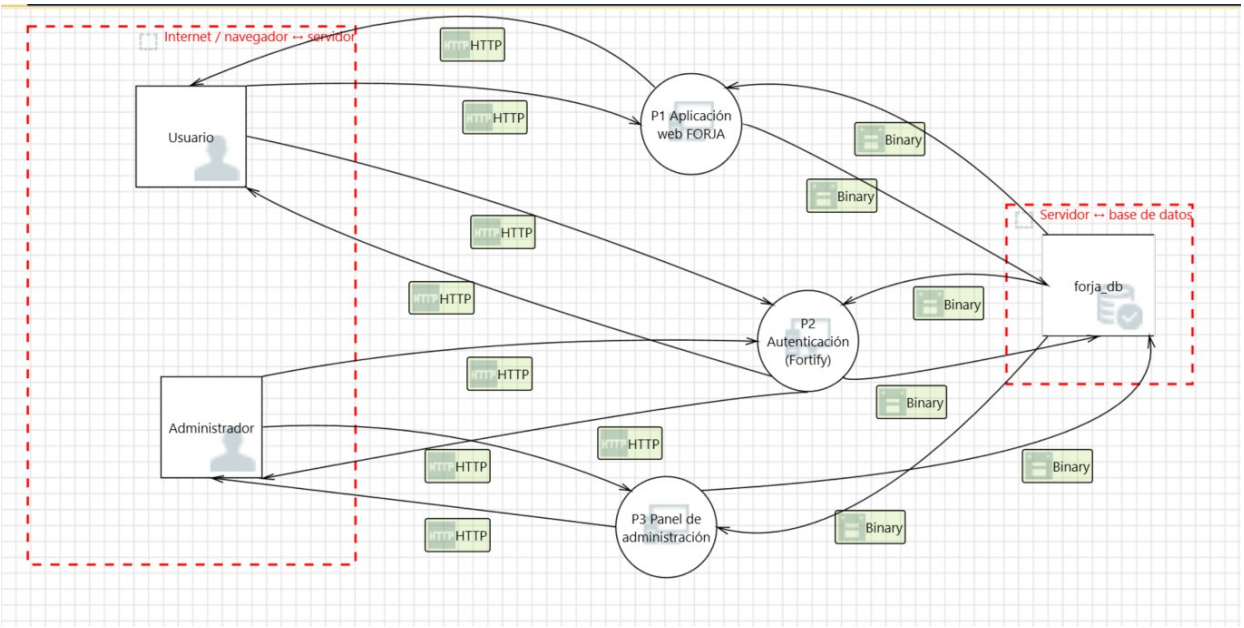


Figura: diagrama de flujo de datos de FORJA (versión 1.0)

Elementos del diagrama:

Tipo	Elemento	Descripción
Actor externo	Usuario	Persona con rol usuario: configura su rutina, registra series y consulta su historial.
Actor externo	Administrador	Personas con rol administrador_global o subadministrador: gestionan usuarios, catálogos y parámetros.
Proceso	P1. Aplicación web FORJA (Laravel + Inertia/React)	Lógica de rutinas, sesiones de entrenamiento, recomendaciones e historial.
Proceso	P2. Autenticación (Laravel Fortify)	Registro, inicio de sesión, verificación de correo, 2FA y passkeys.

Tipo	Elemento	Descripción
Proceso	P3. Panel de administración	Rutas /admin protegidas por los middleware auth, admin y puede-editar; registran auditoría.
Almacén de datos	D1. Base de datos MySQL (forja_db)	Usuarios, rutinas, sesiones de entrenamiento, historial, parámetros, auditoría y sesiones HTTP.
Frontera de confianza	Internet / navegador ↔ servidor	Todo dato que llega del navegador se considera no confiable.
Frontera de confianza	Servidor ↔ base de datos	Acceso interno a los datos mediante las credenciales del archivo .env.

Flujos de datos del diagrama:

Flujo	Origen → destino	Datos
F1	Usuario → P2	Correo, contraseña, código 2FA o passkey
F2	P2 → D1	Consulta de credenciales y creación de sesión
F3	Usuario → P1	Configuración de rutina y registro de series (peso, repeticiones, RPE)
F4	P1 → Usuario	Páginas, rutina, recomendaciones e historial
F5	P1 ↔ D1	Lectura y escritura de rutinas, sesiones e historial
F6	Administrador → P3	Altas, cambios, suspensión de cuentas y parámetros del motor
F7	P3 ↔ D1	Catálogos, permisos y registro de auditoría

3.2 Análisis de la superficie de ataque

Puntos del sistema por los que un atacante podría interactuar con FORJA, identificados a partir del diagrama y de la revisión del código y las rutas:

Categoría	Elemento	Riesgo asociado
Entrada de datos	Formularios de registro e inicio de sesión	Fuerza bruta, relleno de credenciales, contraseñas débiles
Entrada de datos	Formularios de configuración de rutina y registro de series	Inyección, XSS almacenado, datos inválidos
Interfaces	Rutas con identificador (/sesion/{id}, /ejercicios-rutina/{id}, /recomendaciones/{id})	Acceso a datos de otros usuarios (IDOR)

Categoría	Elemento	Riesgo asociado
Interfaces	Panel de administración (/admin/*)	Acceso no autorizado, escalada de privilegios
Autenticación	Sesiones (driver database), 2FA y passkeys con Laravel Fortify	Robo o reutilización de sesión, fuerza bruta
Cabeceras y navegador	Respuestas HTTP y cookies	Clickjacking, XSS, divulgación de la versión de PHP
Datos sensibles	Datos personales y de rendimiento físico	Exposición de información
Infraestructura	Puerto 3306 y credenciales de MySQL	Acceso directo a la base de datos
Dependencias	Paquetes de Composer y npm	Vulnerabilidades conocidas (CVE)
Configuración	Archivo .env y modo debug de Laravel	Fuga de claves e información interna

3.3 Amenazas identificadas, categorización y priorización

Amenazas generadas por la herramienta

Al analizar el diagrama, Microsoft Threat Modeling Tool generó 61 amenazas, todas con prioridad Alta y estado Not Started. Se repiten porque la herramienta crea una por cada flujo y elemento. Se agruparon por categoría STRIDE y se consolidaron en las amenazas de la tabla siguiente.

Categoría STRIDE	Cantidad	Ejemplos de la herramienta	Consolidada en
Spoofing	14	Suplantación de Usuario, Administrador, de los procesos P1, P2 y P3 y de forja_db	T-01, T-02, T-11
Tampering	10	SQL Injection (3), Cross Site Scripting y XSS persistente (4), corrupción de forja_db (3)	T-06, T-07, T-11
Repudiation	9	Un elemento niega haber recibido o escrito datos	T-13
Information Disclosure	5	Control de acceso débil sobre forja_db (2), Data Flow Sniffing (3)	T-03, T-11, T-14
Denial of Service	19	Flujos interrumpidos, almacén inaccesible, caída del proceso, consumo excesivo de recursos	T-12
Elevation of Privilege	4	Ejecución remota de código y cambio del flujo de ejecución en P1 y P3	T-04, T-08
Total	61		

La herramienta no conoce la lógica ni la configuración de FORJA. Por eso el equipo agregó, a partir de la revisión del código y de los resultados de SAST y DAST, amenazas que no genera: acceso a datos de otro usuario por identificador (T-03), cookies de sesión (T-02), .env y modo debug (T-05), dependencias vulnerables (T-08), cabeceras HTTP (T-09) y CSRF (T-10).

Amenazas consolidadas y priorización

Las amenazas se clasificaron con la metodología STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege). La prioridad resulta de combinar probabilidad e impacto, evaluados como Baja (1), Media (2) o Alta (3); riesgo = probabilidad × impacto: Alto de 6 en adelante, Medio de 3 a 5 y Bajo de 1 a 2. Las decisiones posibles son mitigar, aceptar, transferir o evitar.

ID	Amenaza	STRIDE	Prob.	Imp.	Riesgo	Decisión y mitigación
T-01	Fuerza bruta o relleno de credenciales sobre el inicio de sesión	S	2	3	6 Alto	Mitigar. Fortify limita el inicio de sesión a 5 intentos por minuto; existen 2FA y passkeys como segundo factor.
T-02	Robo o reutilización de la cookie de sesión	S / I	2	3	6 Alto	Mitigar. La cookie laravel-session lleva HttpOnly; se agregó el atributo Secure a las cookies del frontend y `SESSION_SECURE_COOKIE=true` queda documentado para producción (3.5).
T-03	Acceso a datos de otro usuario cambiando un identificador en la URL (IDOR)	I	2	3	6 Alto	Mitigar. Los controladores comprueban que el recurso pertenezca al usuario autenticado y responden 403 en caso contrario.
T-04	Usuario sin privilegios accede al panel de administración o edita catálogos	E	2	3	6 Alto	Mitigar. Las rutas /admin usan los middleware auth y admin; las operaciones de escritura exigen además puede-editar (permisos por módulo).
T-05	Exposición de .env o modo debug activo en producción	I	2	3	6 Alto	Mitigar. .env está excluido de Git y nunca se subió; para producción se documenta `APP_DEBUG=false` y `APP_ENV=production` (3.5).

ID	Amenaza	STRIDE	Prob.	Imp.	Riesgo	Decisión y mitigación
T-06	Inyección SQL en formularios y filtros	T	1	3	3 Medio	Mitigar. Todo el acceso a datos usa Eloquent con consultas parametrizadas; no hay consultas SQL crudas en el código.
T-07	XSS almacenado o reflejado	T / I	2	2	4 Medio	Mitigar. React escapa el contenido por defecto; se agregó una Content-Security-Policy con nonce en los scripts.
T-08	Dependencia con vulnerabilidad conocida	T / I	2	2	4 Medio	Mitigar. Snyk Open Source las detectó y se actualizaron los paquetes afectados (sección 4).
T-09	Clickjacking y divulgación de la versión de PHP por cabeceras HTTP ausentes	T / I	2	2	4 Medio	Mitigar. El middleware SecurityHeaders agrega X-Frame-Options, frame-ancestors, nosniff y retira X-Powered-By.
T-10	Falsificación de petición entre sitios (CSRF)	T	2	2	4 Medio	Mitigar. Laravel exige el token CSRF en las peticiones POST, PUT y DELETE.
T-11	Acceso directo a MySQL con el usuario root	I / E	1	3	3 Medio	Mitigar en el despliegue. En producción usar un usuario con privilegios mínimos y no exponer el puerto 3306. En el ambiente local se acepta.
T-12	Denegación de servicio por peticiones masivas	D	2	2	4 Medio	Aceptar en esta etapa. Solo el inicio de sesión tiene límite de intentos; el control general se delega al servidor o al proveedor de hospedaje.
T-13	Acciones administrativas sin trazabilidad (repudio)	R	1	2	2 Bajo	Mitigar. Las acciones administrativas se registran en las pantallas de auditoría e historial de cambios.
T-14	Interceptación del tráfico entre la aplicación y MySQL (Data Flow Sniffing)	I	1	2	2 Bajo	Aceptar en el ambiente local, donde la aplicación y la base de datos están en la misma máquina. En el despliegue, usar una conexión cifrada (TLS) si están en servidores distintos.

3.4 Versionado del modelo de amenazas

El modelo se guardó en tres versiones. Cada una refleja el estado de las amenazas después de una fase del proyecto.

Versión	Cambios en el modelo	Amenazas mitigadas
1.0	Modelo inicial: el diagrama de flujo de datos y las 61 amenazas generadas por la herramienta, todas sin atender.	Ninguna
1.1	Se evalúan las amenazas contra los controles existentes del código y las remediaciones SAST (dependencias actualizadas y atributo Secure en las cookies). La herramienta marca 12 amenazas como mitigadas.	T-01, T-03, T-04, T-06, T-08, T-10, T-13 (T-02 parcial)
1.2	Tras el análisis DAST: se agrega el middleware de cabeceras de seguridad (CSP, clickjacking, nosniff, X-Powered-By) y se documenta la configuración de producción. La herramienta marca 16 amenazas como mitigadas.	T-02, T-05, T-07, T-09

Estado de cada amenaza por versión:

ID	v1.0	v1.1	v1.2
T-01	Sin atender	Mitigada	Mitigada
T-02	Sin atender	Parcial	Mitigada (Secure en producción)
T-03	Sin atender	Mitigada	Mitigada
T-04	Sin atender	Mitigada	Mitigada
T-05	Sin atender	Abierta	Mitigada (documentada para producción)
T-06	Sin atender	Mitigada	Mitigada
T-07	Sin atender	Abierta	Mitigada
T-08	Sin atender	Mitigada	Mitigada
T-09	Sin atender	Abierta	Mitigada
T-10	Sin atender	Mitigada	Mitigada
T-11	Sin atender	Abierta	Abierta (se mitiga en el despliegue)
T-12	Sin atender	Aceptada	Aceptada
T-13	Sin atender	Mitigada	Mitigada
T-14	Sin atender	Aceptada	Aceptada (local)

3.5 Configuración segura para el despliegue (documentada, no aplicada)

El archivo .env guarda la configuración y los secretos de cada ambiente. La configuración actual corresponde al ambiente local de desarrollo. Los valores siguientes se documentan para el momento del despliegue y no se aplicaron al ambiente actual, por lo que no afectan lo ya implementado.

Ajuste	Valor actual (local)	Valor para producción	Motivo
APP_ENV	local	production	Activa el comportamiento de producción del framework.
APP_DEBUG	true	false	Con true, un error muestra rutas, código y configuración a quien lo provoque.
SESSION_SECURE_COOKIE	no definido	true	La cookie de sesión solo viaja por HTTPS; requiere que el sitio se sirva con HTTPS.

Controles que ya existen en el proyecto:

- El archivo .env está excluido del repositorio mediante .gitignore (junto con .env.backup y .env.production).
- El historial de Git no contiene ningún commit que haya incluido el archivo .env.
- El archivo .env.example, que sí se versiona, no contiene secretos reales.

Mejora pendiente: el README del repositorio de FORJA no explica cómo crear y configurar el archivo .env a partir de .env.example.

4. Análisis estático (SAST)

4.1 Herramienta y configuración

Se utilizó Snyk, conectado al repositorio de GitHub del proyecto, y la CLI de Snyk para exportar los resultados completos a archivos JSON (snyk-code.json y snyk-deps.json, guardados en la carpeta /sast). Snyk analizó tres componentes: las dependencias de npm (package.json y package-lock.json), las dependencias de PHP (composer.lock) y el código fuente propio mediante Snyk Code.

Snyk Code corresponde al análisis estático del código (SAST). El análisis de dependencias (Snyk Open Source) revisa las librerías de terceros en busca de vulnerabilidades conocidas y complementa el SAST.

4.2 Resultados del escaneo inicial

Escaneo realizado el 04/11/2026, antes de aplicar cualquier corrección. Total de hallazgos: 31 (1 crítico, 3 altos, 9 medios y 18 bajos).

Componente	Tipo de análisis	Crítica	Alta	Media	Baja
package.json	Dependencias (npm)	1	1	4	1
composer.lock	Dependencias (PHP)	0	2	1	0
Code analysis	Código fuente (SAST)	0	0	4	17
Total		1	3	9	18

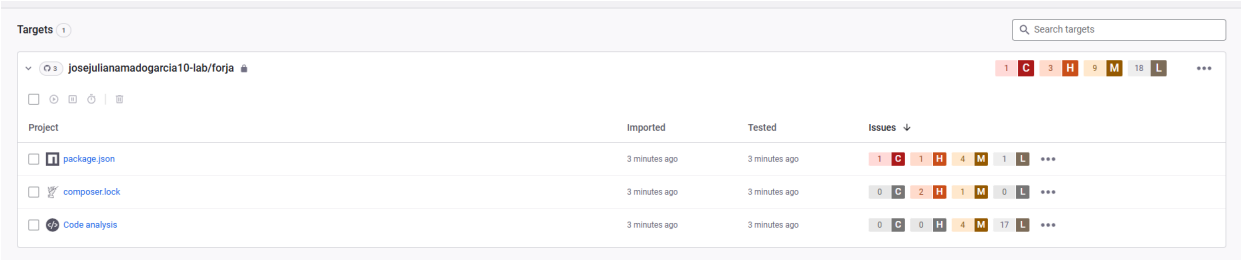


Figura 1. Resultado inicial de Snyk para el repositorio forja (04/11/2026).

La CLI de Snyk revisó 7 proyectos (el proyecto y los manifiestos incluidos en la carpeta vendor). Solo 2 contenían rutas vulnerables: composer.lock y package-lock.json del proyecto; los otros 5 no tienen vulnerabilidades.

El único hallazgo crítico proviene de una dependencia, no del código propio. El análisis de código no arrojó hallazgos críticos ni altos: sus 21 hallazgos son medios y bajos; 19

resultaron falsos positivos y 2 (cookies sin atributo Secure) se corrigieron (secciones 4.3.3 y 4.3.4).

4.3 Detalle de vulnerabilidades

Para cada hallazgo se explica qué significa, cómo podría explotarse y qué impacto tendría. Los identificadores S-01 a S-31 se usan en todo el documento.

4.3.1 Vulnerabilidades críticas y altas (dependencias)

ID	Severidad	Paquete y ruta	Vulnerabilidad	Explicación	Corrección
S-01	Crítica	shell-quote@1.9.0 (vía concurrently @10.0.5)	Command Injection · CWE-78 · CVE-2026-102422 · CVSS 9.2	La función de citado del paquete no trata bien ciertos tokens de comentario. Si un atacante controla el texto que se cita, parte de la cadena se interpreta como comandos del shell y puede ejecutar comandos en el sistema. Existe prueba de concepto pública.	Actualizar concurrently a 10.0.6 (incluye shell-quote 1.11.0)
S-02	Alta	vite@8.2.2 (dependencia directa)	Arbitrary Command Injection · CWE-77 · CVE-2026-112942 · CVSS 7.5	Si un atacante influye en el archivo que recibe la utilidad que abre el editor desde el servidor de desarrollo, puede evadir la validación de rutas UNC en Windows y lograr que se abra un archivo arbitrario, con acceso de lectura y escritura a archivos sensibles.	Actualizar vite a 8.2.4
S-03	Alta	league/commonmark@2.10.1 (vía laravel/framework@13.31.0)	Inefficient Algorithmic Complexity · CWE-400, CWE-407 · CVE-2026-105830 · CVSS 8.7	El analizador de tablas de Markdown (TableStartParser::tryStart) tarda demasiado con un párrafo largo de líneas sin barras verticales ni líneas en blanco que no empiecen con letra. Quien envíe ese texto a un servicio que procese Markdown no confiable puede agotar la CPU y dejarlo sin servicio. En FORJA el riesgo real es bajo: el código propio no procesa Markdown y la librería llega solo como dependencia de laravel/framework.	Actualizar league/commonmark a 2.10.2

ID	Severidad	Paquete y ruta	Vulnerabilidad	Explicación	Corrección
S-04	Alta	symfony/http-foundation@8.1.6 (vía laravel/framework@13.31.0)	ReDoS (Regular Expression Denial of Service) · CWE-1333 · CVE-2026-108082 · CVSS 8.7	La función HeaderUtils::split() usa una expresión regular ambigua. Una cabecera Cache-Control con una larga cadena de espacios provoca retroceso exponencial (2 elevado a N combinaciones) y agota el límite del motor de expresiones regulares. Como Symfony lee esa cabecera en cada petición, basta una petición HTTP con esa cabecera para hacer fallar su procesamiento (denegación de servicio).	Actualizar symfony/http-foundation a 8.1.7

4.3.2 Vulnerabilidades medias (dependencias)

ID	Severidad	Paquete y ruta	Vulnerabilidad	Explicación	Corrección
S-05	Media	vite@8.2.2	Arbitrary Argument Injection · CWE-88 · CVE-2026-112944 · CVSS 6.3	Un nombre de archivo que empieza con guion lo interpreta el editor como opción de línea de comandos y no como ruta. Solo es explotable en entornos WSL.	Actualizar vite a 8.2.4
S-06	Media	vite@8.2.2	Command Injection · CWE-78 · CVE-2026-112943 · CVSS 6.3	En Windows, los caracteres % de las rutas se expanden como variables de entorno en CMD antes de ejecutarse, lo que permite inyectar comandos del sistema. Solo afecta a Windows.	Actualizar vite a 8.2.4
S-07	Media	vite@8.2.2	Directory Traversal · CWE-22 · CVE-2026-112946 · CVSS 6.3	Los módulos resueltos se registran como seguros usando su forma de URL y no la ruta limpia. Así, un archivo fuera de lo permitido puede quedar marcado como seguro y servirse a cualquiera con acceso de red al servidor de desarrollo.	Actualizar vite a 8.2.4

ID	Severidad	Paquete y ruta	Vulnerabilidad	Explicación	Corrección
S-08	Media	vite@8.2.2	Directory Traversal · CWE-22 · CVE-2026-112947 · CVSS 6.3	El parámetro ?vite-wasm-instance no se valida contra la lista de rutas permitidas. Se puede pedir un archivo fuera de ellas, incluidos archivos sensibles como .env.	Actualizar vite a 8.2.4
S-09	Media	league/commonmark@2.10.1 (vía laravel/framework@13.31.0)	Cross-site Scripting (XSS) · CWE-79 · CVE-2026-105829 · CVSS 5.1	La extensión DisallowedRawHtml no escapa bien una etiqueta no permitida que aparece al final de un literal HTML, y un atacante puede inyectar scripts con Markdown manipulado. Solo es explotable si html_input está en allow, la extensión está activa y usuarios no confiables pueden publicar Markdown. En FORJA el código propio no usa Markdown, por lo que el riesgo real es bajo.	Actualizar league/commonmark a 2.10.2

4.3.3 Vulnerabilidades medias del código fuente (Snyk Code)

Las cuatro son la misma regla, Use of Hardcoded Passwords (CWE-798, CWE-259). Esta regla busca contraseñas o claves escritas directamente en el código, que cualquier persona con acceso al repositorio podría leer y usar. Snyk marca todo texto asignado a una clave llamada password; en estos cuatro casos el texto es un mensaje o una etiqueta de Laravel, no una credencial.

ID	Severidad	Archivo y línea	Regla y CWE	Contenido de la línea	Evaluación
S-10	Media	app/Models/User.php, línea 46	Use of Hardcoded Passwords · CWE-798, CWE-259	'password' => 'hashed' (cast del modelo)	Falso positivo: indica que Laravel guarda la contraseña con hash; no es una contraseña.
S-11	Media	lang/es/auth.php, línea 5	Use of Hardcoded Passwords · CWE-798, CWE-259	'password' => 'La contraseña es incorrecta.'	Falso positivo: mensaje de error traducido.
S-12	Media	lang/es/validation.php, línea 12	Use of Hardcoded Passwords · CWE-798, CWE-259	'current_password' => 'La contraseña actual es incorrecta.'	Falso positivo: mensaje de validación traducido.
S-13	Media	lang/es/validation.php, línea 55	Use of Hardcoded Passwords · CWE-798, CWE-259	'password' => 'contraseña'	Falso positivo: nombre traducido del campo.

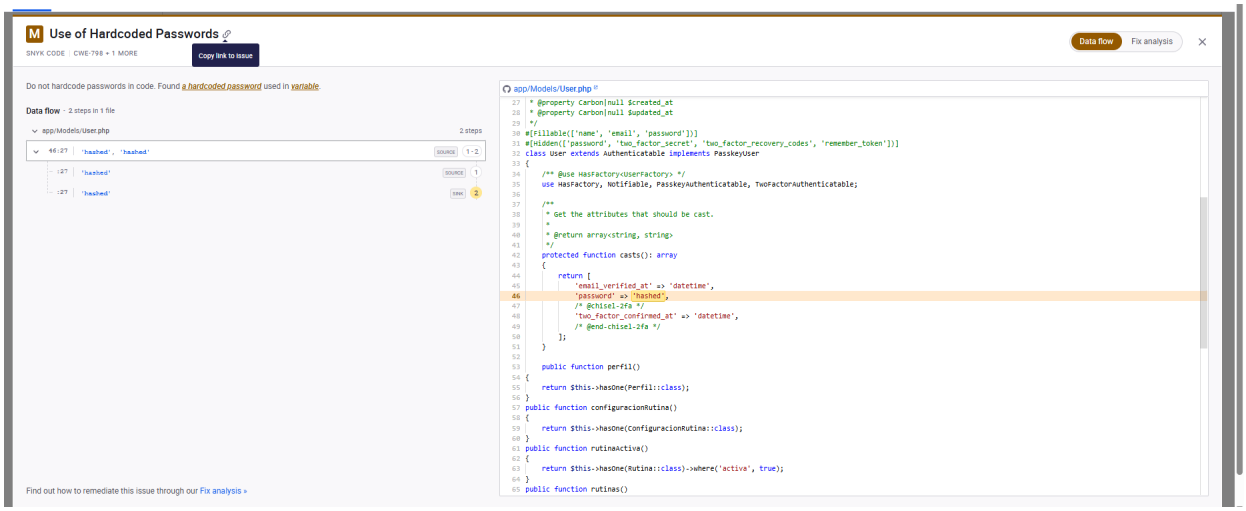


Figura 2. Hallazgo de Snyk Code en app/Models/User.php, línea 46 (falso positivo).

4.3.4 Vulnerabilidades bajas

Son 18: una en dependencias y 17 en código. El detalle línea por línea de las 17 del código está en el Anexo A.

ID	Severidad	Hallazgo	Dónde	Qué significa	Evaluación
S-14	Baja	Cross-site Scripting (XSS) · CWE-80 · CVE-2026-112945 · CVSS 2.1	vite@8.2.2	Las cadenas de consulta de la URL no se eliminan antes de construir el nombre de archivo, lo que puede alterar la resolución de recursos o manipular URLs en el HTML generado.	Se corrige al actualizar vite a 8.2.4.
S-15 y S-16	Baja	Cookie sin atributo Secure · CWE-614	resources/js/components/ui/sidebar.tsx, línea 84; resources/js/hooks/use-appearance.tsx, línea 29	Una cookie sin Secure puede viajar por una conexión HTTP sin cifrar y ser interceptada.	Riesgo bajo: guardan preferencias de interfaz (estado de la barra lateral y tema). Corrección opcional: agregar Secure al servir por HTTPS.
S-17 a S-26	Baja	Use of Hardcoded Passwords en pruebas · CWE-798, CWE-259	10 líneas en tests/Feature/Auth y tests/Feature/Settings	Contraseñas escritas en el código de las pruebas automáticas.	Falso positivo: valores ficticios como wrong-password o new-password que solo se usan en pruebas.
S-27 a S-31	Baja	Hash con esfuerzo computacional insuficiente (MD5 y SHA-1) en pruebas · CWE-916	tests/Feature/Auth/AuthenticationTest.php, línea 83; tests/Feature/Auth/EmailVerificationTest.php, líneas 42, 62, 80 y 110	MD5 y SHA-1 no sirven para proteger contraseñas porque se calculan muy rápido.	Falso positivo: en las pruebas se usan para construir la clave del limitador de intentos y el hash del enlace de verificación de correo; no protegen contraseñas.

4.4 Remediaciones

Se priorizan las vulnerabilidades críticas y altas, luego las medias y bajas de dependencias. Los hallazgos de código, al ser falsos positivos, se documentan y se marcan como ignorados en Snyk con su justificación. Las correcciones se hicieron en una rama independiente (fix/snyk-dependencias) y se integraron a la rama principal mediante un pull request.

ID	Acción	Comando o cambio	Estado
S-01	Actualizar concurrently	`npm install concurrently@^10.0.6`. Resultado: concurrently 10.0.6 y shell-quote 1.12.0	Completada
S-02, S-05 a S-08, S-14	Actualizar vite	`npm install vite@^8.2.4`. Resultado: vite 8.2.4, que es la versión corregida	Completada
S-03, S-04, S-09	Actualizar league/commonmark y symfony/http-foundation	`composer update league/commonmark symfony/http-foundation --with-dependencies`. Resultado: league/commonmark 2.10.7 y symfony/http-foundation 8.1.11	Completada
S-10 a S-13	Marcar como falso positivo (Ignore) en Snyk con justificación escrita	Sin cambio de código. Ignorados en Snyk con su justificación	Completada
S-15 y S-16	Agregar el atributo Secure a las cookies	Se agrega Secure (y SameSite=Lax) a las cookies cuando el sitio se sirve por HTTPS, en use-appearance.tsx y sidebar.tsx. En HTTP no se agrega para no perder la preferencia en desarrollo local	Completada
S-17 a S-31	Marcar como falso positivo (Ignore) por ser código de pruebas	Sin cambio de código. Ignorados en Snyk con su justificación	Completada

Los hallazgos S-15 y S-16 (cookies) no se ignoran porque eran hallazgos reales de bajo riesgo y se corrigieron en el código. Para los falsos positivos se registra en Snyk la siguiente justificación al marcarlos como Ignore:

Hallazgos	Justificación registrada
S-10 a S-13	Falso positivo. La línea contiene un mensaje o una etiqueta traducida, o el cast hashed de Laravel; no hay ninguna contraseña ni credencial.
S-17 a S-26	Falso positivo. Valor ficticio usado únicamente en pruebas automáticas; no es una credencial real.
S-27 a S-31	Falso positivo. MD5 o SHA-1 usados en pruebas para construir la clave de un limitador y el hash de un enlace de verificación; no protegen contraseñas.

Verificación después de cada corrección: compilar el frontend con `npm run build`, ejecutar las pruebas con `php artisan test` y volver a escanear con Snyk.

Cambios realizados (diferencias en package.json y composer.lock):

Paquete	Versión anterior	Versión nueva
concurrently (npm)	^10.0.3	^10.0.6 (instalado 10.0.6, con shell-quote 1.12.0)
vite (npm)	^8.0.0 (instalado 8.2.2)	^8.2.4 (instalado 8.2.4)
league/commonmark (composer)	2.10.1	2.10.7
symfony/http-foundation (composer)	8.1.6	8.1.11
Dependencias de apoyo actualizadas por composer	nette/schema 1.3.6, nette/utils 4.1.5, symfony/deprecation-contracts 3.7.1, symfony/polyfill-mbstring 1.38.2, symfony/polyfill-php80 1.37.0	nette/schema 1.4.0, nette/utils 4.1.6, symfony/deprecation-contracts 3.8.1, symfony/polyfill-mbstring 1.43.3, symfony/polyfill-php80 1.43.0

Después de actualizar, `npm run build` compiló sin errores y `php artisan test` pasó las 39 pruebas (136 aserciones), igual que antes de los cambios. Los archivos modificados fueron package.json, package-lock.json y composer.lock.

4.5 Escaneo posterior a las correcciones

Reescaneo realizado el 4 de noviembre de 2026 con Snyk CLI (`snyk code test` y `snyk test --all-projects`). Resultados comparados con el escaneo inicial:

Severidad	Hallazgos abiertos antes	Hallazgos abiertos después	Diferencia
Crítica	1	0	-1
Alta	3	0	-3
Media	9	0	-9
Baja	18	0	-18
Total	31	0	-31

La tabla cuenta hallazgos abiertos: no significa que los 31 se hayan corregido. Doce se corrigieron y diecinueve se clasificaron como falsos positivos y se marcaron como ignorados en Snyk:

Resultado	Cantidad	Hallazgos
Corregidos	12	10 de dependencias (S-01 a S-09 y S-14) y 2 cookies sin atributo Secure (S-15 y S-16)
Falsos positivos ignorados	19	S-10 a S-13 y S-17 a S-31
Abiertos	0	Ninguno
Total	31	

Los 10 hallazgos de dependencias (Snyk Open Source) desaparecieron al actualizar los paquetes: el escaneo de composer y npm terminó con "no vulnerable paths found". Las dos cookies sin Secure (S-15 y S-16) se corrigieron en el código. Los 19 hallazgos restantes de Snyk Code son falsos positivos y se marcaron como Ignored en Snyk con su justificación escrita (tipo Not vulnerable, sin expiración). Después de un reescaneo, el proyecto forja queda con 0 hallazgos abiertos y 19 ignorados. Después de las correcciones, `npm run build` compiló sin errores y `php artisan test` pasó las 39 pruebas.

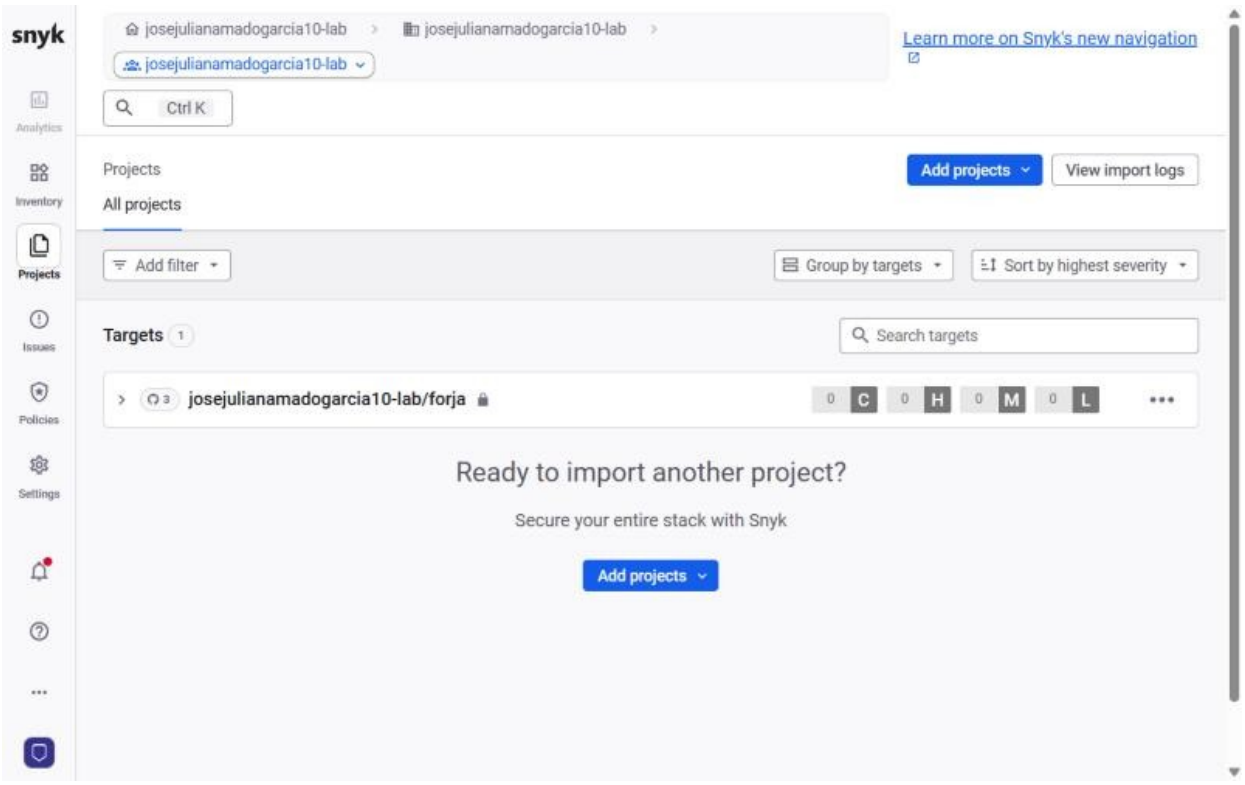


Figura: reporte final de Snyk para el proyecto forja (0 críticas, 0 altas, 0 medias, 0 bajas)

5. Análisis dinámico (DAST)

5.1 Herramienta y configuración

Se utilizó OWASP ZAP 2.17.0 contra la aplicación FORJA en ejecución en un ambiente local de pruebas: http://localhost:8000, servida con `php artisan serve` y con los archivos del frontend compilados (`npm run build`). Antes del escaneo se respaldó la base de datos y se crearon dos cuentas de prueba, una con rol usuario y otra con rol administrador_global.

- Exploración manual con el navegador de ZAP, iniciando sesión con cada una de las dos cuentas y recorriendo todas las pantallas, para registrar las rutas y formularios de cada rol.
- Spider: 152 URLs encontradas y 59 nodos incorporados al sitio.
- Escaneo activo con la política Default Policy: 5,024 peticiones enviadas. Se excluyeron del escaneo las rutas /login, /logout y /settings/profile para no saturar el limitador de intentos, cerrar la sesión ni borrar la cuenta de prueba.
- El escaneo activo se ejecutó sin sesión iniciada; las rutas protegidas se cubrieron con la exploración manual autenticada.

5.2 Resultados del escaneo

Escaneo realizado el 4 de noviembre de 2026. ZAP no encontró vulnerabilidades de riesgo alto ni fallas de inyección (SQL, XSS, comandos) en las pruebas activas. Las alertas son de configuración y de cabeceras de seguridad.

Nivel de riesgo	Cantidad de alertas
Alto	0
Medio	4
Bajo	6
Informativo	7
Total	17

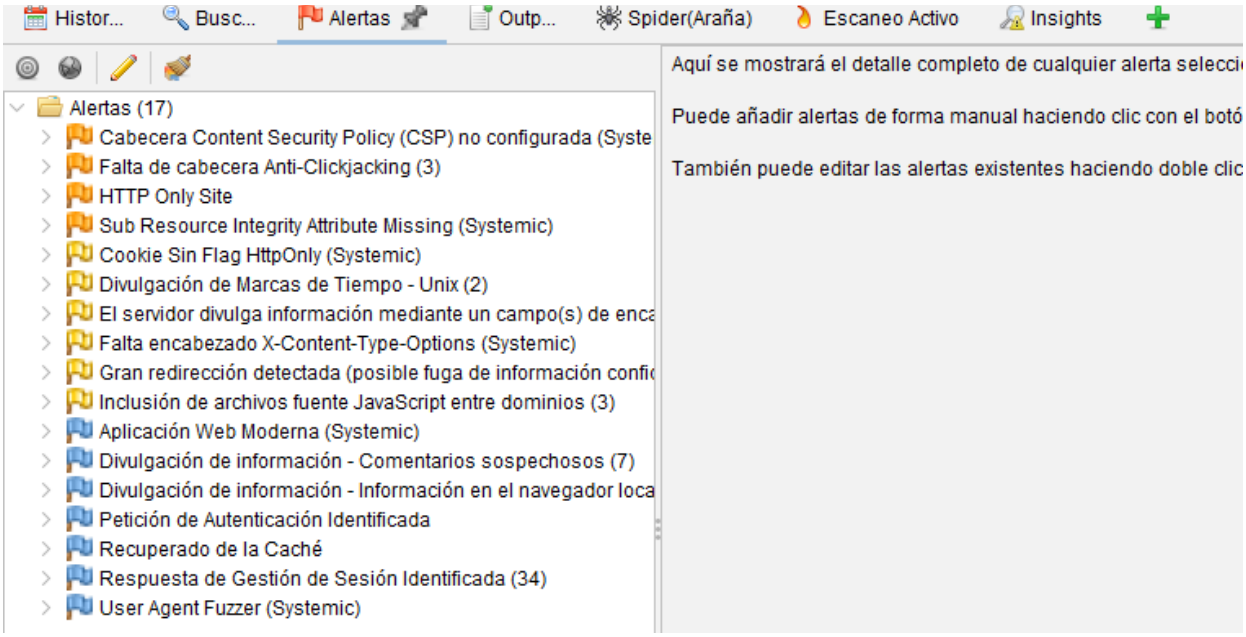


Figura: alertas detectadas por OWASP ZAP (17 tipos de alerta)

5.3 Detalle de vulnerabilidades

Para cada alerta se indica si corresponde a un problema real de la aplicación, a una característica del ambiente de desarrollo o a un falso positivo.

ID	Riesgo	Alerta	Dónde	Explicación y evaluación	Recomendación
D-01	Media	Content Security Policy (CSP) no configurada · CWE-693	http://localhost:8000/ (3 URLs)	Sin CSP, el navegador no limita de dónde se cargan scripts, estilos e imágenes, lo que facilita ataques XSS. Hallazgo real.	Agregar la cabecera Content-Security-Policy con un middleware de Laravel.
D-02	Media	Falta cabecera Anti-Clickjacking · CWE-1021	http://localhost:8000/ (3 URLs)	Otro sitio podría mostrar FORJA dentro de un marco invisible y engañar al usuario para que haga clic. Hallazgo real.	Agregar X-Frame-Options: DENY o la directiva frame-ancestors en la CSP.
D-03	Media	HTTP Only Site · CWE-311	http://localhost:8000	El sitio se sirve sin HTTPS, por lo que el tráfico viaja sin cifrar. Corresponde al ambiente local de desarrollo.	Servir producción solo por HTTPS y activar `SESSION_SECURE_COOKIE=true` (sección 3.5).

ID	Riesgo	Alerta	Dónde	Explicación y evaluación	Recomendación
D-04	Media	Sub Resource Integrity Attribute Missing · CWE-345	http://localhost:8000/ (5 URLs)	Los recursos señalados venían del servidor de desarrollo de Vite (http://[::1]:5173), activo durante la primera parte de la exploración. No existe en el sitio compilado. Falso positivo del ambiente.	Sin acción: en producción los recursos se sirven desde el mismo dominio, ya compilados.
D-05	Baja	Cookie sin HttpOnly · CWE-1004	XSRF-TOKEN en http://localhost:8000/ (5 URLs)	Laravel envía la cookie XSRF-TOKEN sin HttpOnly a propósito, porque el frontend debe leerla para enviar el token CSRF. La cookie de sesión laravel-session sí lleva HttpOnly. Riesgo aceptado.	Sin acción.
D-06	Baja	Divulgación de marcas de tiempo Unix · CWE-497	/build/assets/Mostrar-B_tuI8lQ.js (2 URLs)	Un número dentro del JavaScript compilado coincide con el formato de una fecha. No es información sensible. Falso positivo.	Sin acción.
D-07	Baja	Cabecera X-Powered-By revela la versión de PHP · CWE-497	http://localhost:8000/ (5 URLs)	La respuesta indica PHP/8.4.12, lo que ayuda a un atacante a buscar fallas de esa versión. Hallazgo real de bajo riesgo.	Desactivar con `expose_php=0ff` en php.ini o quitar la cabecera en un middleware.
D-08	Baja	Falta X-Content-Type-Options · CWE-693	http://localhost:8000 (y un recurso de archive.mozilla.org)	Sin la cabecera nosniff, navegadores antiguos pueden interpretar un archivo como otro tipo. Una de las URLs es tráfico propio de Firefox, fuera del alcance. Hallazgo real en la aplicación.	Agregar X-Content-Type-Options: nosniff.
D-09	Baja	Redirección con cuerpo largo · CWE-201	POST /login hacia /dashboard (27 URLs)	La respuesta es la redirección estándar de Laravel con un cuerpo breve de 370 bytes; no contiene datos sensibles. Falso positivo.	Sin acción.
D-10	Baja	Inclusión de JavaScript de otro dominio · CWE-829	http://localhost:8000/ (3 URLs)	El script señalado es http://[::1]:5173/@vite/client, del servidor de desarrollo de Vite. No existe en el sitio compilado. Falso positivo del ambiente.	Sin acción.

ID	Riesgo	Alerta	Dónde	Explicación y evaluación	Recomendación
D-11	Info	Comentarios sospechosos · CWE-615	/build/assets/app-DV8dne38.js (7 URLs)	ZAP encontró la palabra USER dentro del JavaScript minificado. No es un comentario ni contiene datos sensibles. Falso positivo.	Sin acción.
D-12	Info	Otras alertas informativas	Varias	Datos en localStorage (preferencia de tema appearance=system), aplicación web moderna, petición de autenticación identificada, gestión de sesión identificada, recuperado de la caché (tráfico de Firefox) y User Agent Fuzzer. Son avisos para entender la aplicación, no vulnerabilidades.	Sin acción.

Resumen: de las 17 alertas, ninguna es de riesgo alto. Los hallazgos reales y corregibles son cuatro: la falta de CSP (D-01), de protección contra clickjacking (D-02), de nosniff (D-08) y la cabecera X-Powered-By (D-07). Los demás corresponden al ambiente local de desarrollo (HTTP sin cifrar y servidor de Vite), a comportamientos propios de Laravel o a falsos positivos.

5.4 Correcciones y reescaneo

Corrección aplicada: se agregó el middleware App\Http\Middleware\SecurityHeaders, registrado al inicio del grupo web de Laravel. Envía Content-Security-Policy (los scripts solo se aceptan del mismo dominio o con un nonce generado en cada respuesta), X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy y Permissions-Policy, retira la cabecera X-Powered-By y envía HSTS únicamente cuando la conexión es HTTPS. Se acompañó de dos pruebas automáticas (41 pruebas en total, todas pasan) y se integró a la rama principal mediante un pull request.

ID	Corrección	Estado
D-01, D-02	Content-Security-Policy y protección contra clickjacking (frame-ancestors y X-Frame-Options)	Completada en las páginas de la aplicación
D-07	Quitar la cabecera X-Powered-By	Completada en las páginas de la aplicación
D-08	X-Content-Type-Options: nosniff	Completada en las páginas; pendiente en archivos estáticos del servidor de desarrollo
D-03	HTTPS y `SESSION_SECURE_COOKIE=true` en producción	Documentado en la sección 3.5
D-04, D-05, D-06, D-09 a D-12	Falsos positivos, riesgo aceptado o aviso informativo	Sin acción

Resultado del reescaneo

Se repitió el escaneo en una sesión nueva de ZAP, con el mismo procedimiento: exploración manual autenticada, Spider y escaneo activo con Default Policy, sobre el sitio compilado. Para la comparación se consideran solo las alertas de http://localhost:8000. Las alertas de archive.mozilla.org y de https://localhost:8000 corresponden a tráfico propio del navegador de ZAP y se excluyen.

Nivel de riesgo	Antes	Después
Alto	0	0
Medio	4	3
Bajo	6	5
Informativo	7	6
Total	17	14

Alerta	Antes	Después	Evaluación
D-02 Falta cabecera Anti-Clickjacking	3 URLs	0	Corregida.
D-01 CSP no configurada	3 URLs	5 URLs	Corregida en todas las páginas de la aplicación. Las 5 URLs restantes son rutas que Laravel no procesa por el grupo web: /admin y /admin/ejercicios/40 (no existe una ruta GET con ese nombre), /sitemap.xml (no existe) y el directorio /build servido por PHP. Son respuestas de error sin contenido.
D-07 X-Powered-By	5 URLs	5 URLs	Corregida en las páginas. Persiste en las mismas respuestas de error. En producción se resuelve con `expose_php=Off`.
D-08 Falta nosniff	Aplicación y Firefox	5 archivos de /build/assets	Corregida en las páginas. Los archivos estáticos los entrega directamente el servidor de desarrollo de PHP, sin pasar por Laravel; en producción los entrega el servidor web.
CSP: style-src unsafe-inline	No existía	2 URLs (medio)	Alerta nueva, consecuencia de la política. Se permite el estilo en línea porque Inertia y los componentes de React insertan elementos de estilo en tiempo de ejecución. Riesgo aceptado: los scripts siguen restringidos a nonce, que es la protección contra XSS.
D-04 y D-10 (servidor de Vite)	5 y 3 URLs	0	Desaparecen porque el reescaneo se hizo solo con el sitio compilado.
D-03, D-05, D-06, D-09, D-11, D-12	Sin cambio	Sin cambio	Siguen las mismas evaluaciones de la sección 5.3.

Conclusión: no hay alertas de riesgo alto, y los hallazgos reales D-01, D-02, D-07 y D-08 quedaron corregidos en todas las páginas de la aplicación. Los residuos que reporta ZAP son de tres tipos: respuestas de error y archivos estáticos que entrega el servidor de desarrollo, una alerta nueva por permitir estilos en línea (riesgo aceptado) y el uso de HTTP del ambiente local. Una mejora posible, no aplicada por el límite de tiempo del proyecto, es registrar el middleware como global para que también cubra las respuestas de error.

6. Relación con el modelo de amenazas

Coherencia entre los hallazgos de SAST y DAST y las amenazas identificadas en el modelo:

Amenaza (modelo de amenazas)	Hallazgo relacionado	Resultado
T-01 Fuerza bruta en el login	Ninguno. El escaneo activo excluyó /login para no agotar el límite de 5 intentos; S-17 a S-19 son contraseñas de prueba, falsos positivos.	No detectada
T-02 Robo de cookie de sesión	S-15 y S-16 (cookie sin Secure); D-03 (HTTP) y D-05 (XSRF-TOKEN sin HttpOnly).	Mitigada; D-05 es riesgo aceptado
T-03 IDOR	Sin hallazgos en Snyk ni en ZAP.	No detectada
T-05 .env y modo debug	D-03; sin hallazgos de Snyk.	Mitigada (documentada)
T-06 Inyección SQL	Sin hallazgos en Snyk ni en las pruebas activas de ZAP.	No detectada
T-07 XSS	D-01 (sin CSP). Sin fallas de XSS en las pruebas activas.	Mitigada con CSP
T-08 Dependencias vulnerables	Snyk Open Source (sección 4).	Mitigada
T-09 Cabeceras HTTP	D-01, D-02, D-07 y D-08.	Confirmada y mitigada

7. Conclusiones y lecciones aprendidas

- Integrar seguridad desde el diseño ahorra trabajo: el modelo de amenazas señaló qué revisar antes de escanear, y al escanear ya sabíamos qué hallazgos importaban y cuáles eran ruido del ambiente.
- SAST (Snyk) encontró problemas en el código y en las dependencias sin ejecutar la aplicación; DAST (ZAP) encontró problemas de configuración visibles solo con la aplicación en marcha, como las cabeceras HTTP. Ninguna de las dos herramientas encontró fallas de inyección, y cada una vio lo que la otra no.
- Se aceptan la cookie XSRF-TOKEN sin HttpOnly (el frontend necesita leerla), los estilos en línea en la CSP (Inertia y React los generan al ejecutarse), la falta de límite general de peticiones y el uso de MySQL con root en el ambiente local.
- Registrar el middleware de cabeceras como global, desplegar con HTTPS y la configuración de la sección 3.5, usar un usuario de base de datos con privilegios mínimos, limitar la tasa de peticiones y ejecutar Snyk y ZAP de forma automática en cada cambio.

Anexo A. Hallazgos bajos del código fuente (Snyk Code)

Detalle de las 17 vulnerabilidades bajas del análisis de código, agrupadas en 4.3.4.

ID	Archivo	Línea	Regla
S-15	resources/js/components/ui/sidebar.tsx	84	Cookie sin atributo Secure (CWE-614)
S-16	resources/js/hooks/use-appearance.tsx	29	Cookie sin atributo Secure (CWE-614)
S-17	tests/Feature/Auth/AuthenticationTest.php	62	Use of Hardcoded Passwords (CWE-798)
S-18	tests/Feature/Auth/AuthenticationTest.php	87	Use of Hardcoded Passwords (CWE-798)
S-19	tests/Feature/Auth/PasswordResetTest.php	89	Use of Hardcoded Passwords (CWE-798)
S-20	tests/Feature/Auth/PasswordResetTest.php	90	Use of Hardcoded Passwords (CWE-798)
S-21	tests/Feature/Settings/ProfileUpdateTest.php	90	Use of Hardcoded Passwords (CWE-798)
S-22	tests/Feature/Settings/SecurityTest.php	102	Use of Hardcoded Passwords (CWE-798)
S-23	tests/Feature/Settings/SecurityTest.php	103	Use of Hardcoded Passwords (CWE-798)
S-24	tests/Feature/Settings/SecurityTest.php	121	Use of Hardcoded Passwords (CWE-798)
S-25	tests/Feature/Settings/SecurityTest.php	122	Use of Hardcoded Passwords (CWE-798)
S-26	tests/Feature/Settings/SecurityTest.php	123	Use of Hardcoded Passwords (CWE-798)
S-27	tests/Feature/Auth/AuthenticationTest.php	83	Hash inseguro, MD5 (CWE-916)
S-28	tests/Feature/Auth/EmailVerificationTest.php	42	Hash inseguro, SHA-1 (CWE-916)
S-29	tests/Feature/Auth/EmailVerificationTest.php	62	Hash inseguro, SHA-1 (CWE-916)
S-30	tests/Feature/Auth/EmailVerificationTest.php	80	Hash inseguro, SHA-1 (CWE-916)
S-31	tests/Feature/Auth/EmailVerificationTest.php	110	Hash inseguro, SHA-1 (CWE-916)

Anexo B. Glosario

Término	Definición
Secure SDLC	Integración de la seguridad en cada fase del ciclo de vida del software.
Modelo de amenazas	Representación del sistema para identificar amenazas, su probabilidad e impacto.
STRIDE	Clasificación de amenazas: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.
Superficie de ataque	Conjunto de puntos por los que un atacante puede interactuar con el sistema.
SAST	Análisis del código fuente sin ejecutarlo (caja blanca).
DAST	Prueba de la aplicación en ejecución desde el exterior (caja negra).
SCA	Análisis de dependencias de terceros en busca de vulnerabilidades conocidas.
Falso positivo	Hallazgo que la herramienta reporta pero que no representa un riesgo real.
CVE	Identificador público de una vulnerabilidad conocida.
CWE	Categoría de debilidad de software (por ejemplo, CWE-89: inyección SQL).
Remediación	Corrección de una vulnerabilidad y verificación de que desapareció.

Anexo C. Evidencias

Capturas del escaneo inicial de Snyk (04/11/2026), antes de aplicar correcciones.

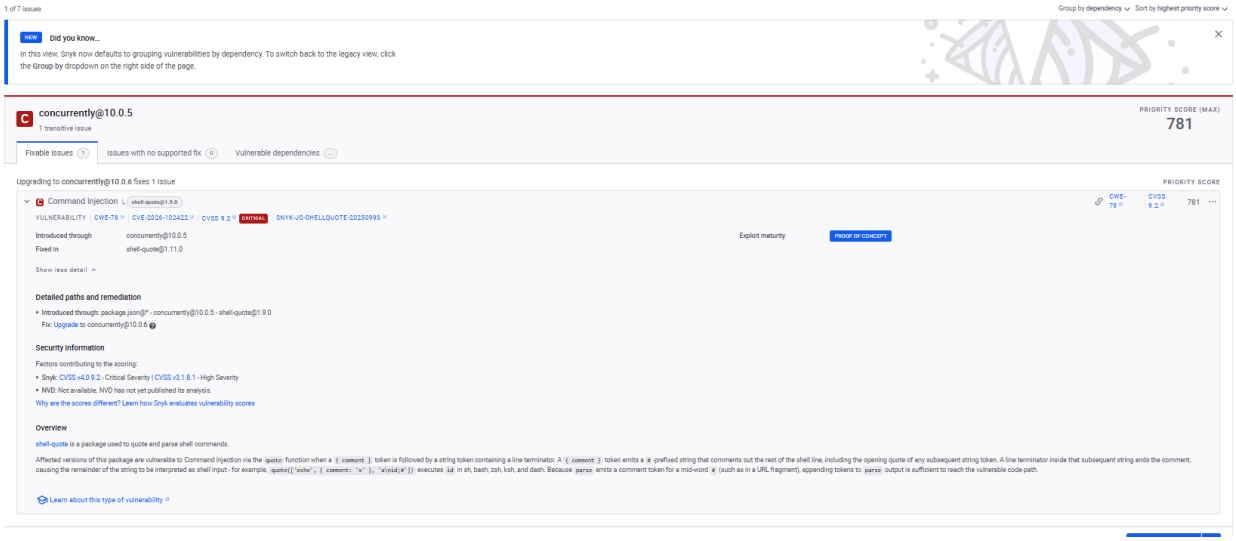


Figura 3. Hallazgo crítico S-01: Command Injection en shell-quote, introducido por concurrently@10.0.5.

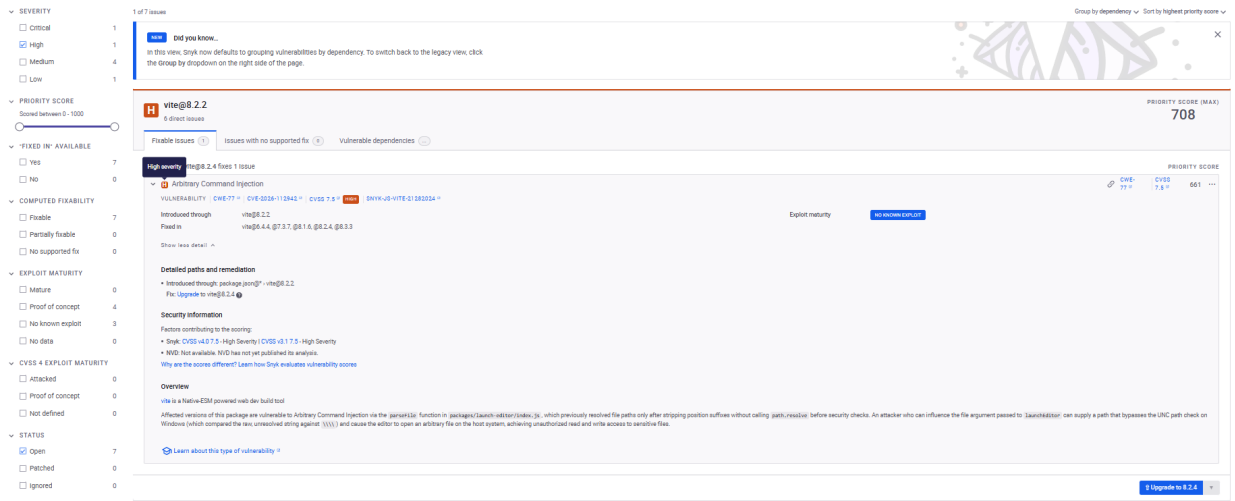


Figura 4. Hallazgo alto S-02: Arbitrary Command Injection en vite@8.2.2.